AWS EC2 + Application Load Balancer Assignment

Topic: EC2 Web Servers, ALB, Health Checks and High Availability

1. Launch Two EC2 Instances and Install Web Server

Objective: Launch two Amazon Linux 2 EC2 instances and run a simple HTTP web server on both.

Steps:

• Open AWS Management Console → EC2 → Instances → Launch Instance.

• Select **Amazon Linux 2** AMI and create two instances, for example **WebServer-1** and **WebServer-2**.

• Allow inbound **HTTP (TCP 80)** and SSH (TCP 22) in the Security Group. For better security, SSH should ideally be restricted to your IP.

• Connect to each instance using SSH and install/start Apache httpd.

Commands on each server:

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl enable --now httpd
```

Create different home pages so that the two servers can be identified:

```
echo "Hello from Web Server 1" | sudo tee /var/www/html/index.html
For Server 2, use: echo "Hello from Web Server 2" | sudo tee /var/www/html/index.html
```

Verification: Open each instance's Public IPv4 address in a browser. Both pages should return HTTP content successfully.

2. Create an Application Load Balancer

Steps: EC2 → Load Balancers → Create Load Balancer → Application Load Balancer.

• Give the ALB a name such as **InstaVibe-ALB**.

• Choose **Internet-facing** and IPv4.

• Select at least two Availability Zones/subnets.

• Create or select a Security Group that allows HTTP traffic on port 80.

• Create a Target Group for **Instances**, protocol HTTP, port 80.

• Register WebServer-1 and WebServer-2 as targets.

• Create the ALB listener: HTTP : 80 → forward to the target group.

Testing: Copy the ALB's DNS name from the Load Balancer details and open it in a browser. Refresh the page several times. The response can come from Web Server 1 or Web Server 2, demonstrating load distribution.

3. Configure a Custom Health Check: /health

On both EC2 instances, create a health endpoint that returns HTTP 200 OK:

```
echo "OK" | sudo tee /var/www/html/health
```

In the Target Group → Health checks, set **Protocol = HTTP** and **Path = /health**. Keep the port as traffic port unless your setup requires another port.

Verification: Open http://<server-public-ip>/health and confirm that the response is **OK**. The target group should show both instances as **Healthy**. If an instance does not return a successful health check, the ALB removes it from the healthy target set and stops routing new requests to it.

4. Simulate Server Failure and Observe High Availability

Stop one EC2 instance from the EC2 console. After the ALB health check detects that the stopped instance is unhealthy, the ALB stops sending new requests to that instance and continues routing requests to the remaining healthy instance.

Observation: The target group changes the stopped server to an unhealthy/unavailable state, while the other server remains healthy. The ALB DNS name can continue serving the website through the remaining instance. This demonstrates **high availability** because traffic can continue even when one backend server fails.

Note: The exact detection time depends on the configured health-check interval and unhealthy threshold.

5. ALB vs NLB vs Classic Load Balancer

Point	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Protocols	HTTP, HTTPS; Layer 7	TCP, TLS, UDP, QUIC; Layer 4	HTTP/HTTPS and TCP; older Layer 4/7 model
Use cases	Web apps, REST APIs, microservices, path/host-based routing	Very high-performance or low-latency TCP/UDP/TLS applications	Legacy applications that still use the older ELB architecture
Features	Content-based routing, host/path rules, HTTP-aware health checks, redirects, authentication integrations	Very high throughput, static IPs per AZ, TLS termination, TCP/UDP/TLS listeners	Basic load balancing and health checks; fewer modern routing features
Layer	Layer 7 (Application)	Layer 4 (Transport)	Primarily Layer 4 with HTTP/HTTPS support

Conclusion

Two EC2 web servers were placed behind an Application Load Balancer. The ALB distributes HTTP traffic among healthy targets, uses the /health endpoint to check server health, and automatically stops routing traffic to a failed instance. This setup improves availability and provides basic fault tolerance for the web application.

Submission tip: Add screenshots of the two EC2 instances, both public IP pages, Target Group health status, ALB DNS page, and the browser result after stopping one instance if your instructor requires practical evidence.